

Computación Paralela y Distribuida

2026-II

José Fiestas PhD

26 de agosto de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Objetivos:

1. Velocidad, eficiencia, escalabilidad. Ley de Ahmdal
2. Speedup, performance
3. Modelos computacionales en paralelo - PRAM (1)
4. **Modelos computacionales en paralelo - PRAM (2)**
5. Particionamiento de datos y tareas
6. Random en paralelo
7. N-body, Divide y Vencerás (mergesort)

Al finalizar la sesión, los alumnos conocen:

1. Modelos PRAM de Suma de Prefijos, Matrices

PRAM: Suma de prefijos (algoritmo no-recursive)

Algorithm 1: Suma de prefijos (algoritmo no-recursive)

```
1 Input: vector A con  $n = 2^k$  números
2 Output: vector C /  $C(0,j)$  es la j-suma de prefijos para  $1 \leq j \leq n$ 
3 for  $1 \leq j \leq n$  pardo
4    $B(0,j) \leftarrow A(j)$ 
5 for  $h = 1$  to  $\log(n)$  do
6   for  $1 \leq j \leq n/2^h$  pardo
7      $B(h,j) \leftarrow B(h-1, 2j-1) \cdot B(h-1, 2j)$ 
8 for  $h = \log(n)$  to 0 do
9   for  $1 \leq j \leq n/2^h$  pardo
10    if  $j$  is even then
11       $C(h,j) \leftarrow C(h+1, j/2)$ 
12    else if  $j = 1$  then
13       $C(h,1) \leftarrow B(h,1)$ 
14    else
15       $C(h,j) \leftarrow C(h+1, (j-1)/2) \cdot B(h,j)$ 
```

PRAM: Suma de prefijos

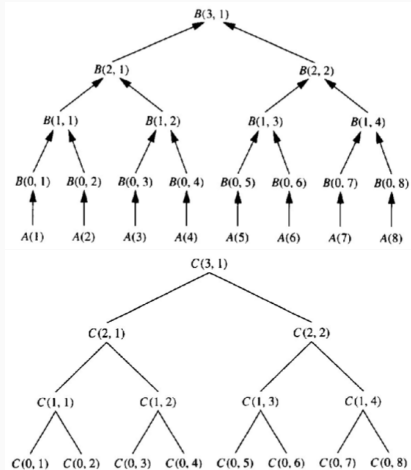


Figura 1: Suma de prefijos de 8 elementos [1] (Fig.2.2-2.3 p.48-49)

PRAM: Suma de prefijos

En el paso 1 se ejecutan n operaciones

En el paso 2 se ejecutan $\sum_{m=1}^k W_{2,m}$ operaciones. $W_{2,m} = n/2^m = 2^{k-m}$, $1 \leq m \leq k$

En el paso 3 se ejecuta $\sum_{m=0}^k W_{3,m}$ operaciones. $W_{3,m} = 2^m$, $0 \leq m \leq k$

Por lo tanto:

$$W(n) = W_1(n) + W_2(n) + W_3(n) = n + 2^k \sum_{m=1}^k 2^{-m} + \sum_{m=0}^k 2^m$$

$$= n + n(1 - (1/n)) + 2n - 1 = O(n)$$

$$T(n) = 2O(\log(n)) + 1 = O(\log(n))$$

Suma de Prefijos (n,p)

Algorithm 2: Suma de Prefijos, algoritmo por proceso ($p < n$)

```
1 Input: vector  $A$  with  $n = 2^k$  numbers,  $p = 2^q \leq n$  processes, process ID  $v$  with  
    $1 \leq v \leq p$   
2 Output:  $C(0, j)$  as prefix products for  $j$  in segment  $\left[\frac{n}{p}(v-1) + 1, \frac{n}{p}v\right]$   
3 Process  $p_s$  ejecuta:  
4 for  $j = 1$  to  $n/p$  do  
5    $B(0, \frac{n}{p}(v-1) + j) \leftarrow A(\frac{n}{p}(v-1) + j)$   
6 for  $h = 1$  to  $\log(n)$  do  
7   if  $k - h - q \geq 0$  then  
8     for  $j = \frac{n}{p2^h}(v-1) + 1$  to  $\frac{n}{p2^h}v$  do  
9        $B(h, j) \leftarrow B(h-1, 2j-1) \cdot B(h-1, 2j)$   
10  else if  $v \leq n/p$  then  
11     $B(h, v) \leftarrow B(h-1, 2v-1) \cdot B(h-1, 2v)$ 
```

Suma de Prefijos (n,p)

Algorithm 3: Suma de Prefijos, algoritmo por proceso ($p < n$)

```
1  for  $h = \log(n)$  to 0 do
2      if  $k - h - q \geq 0$  then
3          for  $j = \frac{n}{p2^h}(v-1) + 1$  to  $\frac{n}{p2^h}v$  do
4              if  $j$  even then
5                   $C(h, j) \leftarrow C(h+1, j/2)$ 
6              else if  $j = 1$  then
7                   $C(h, 1) \leftarrow B(h, 1)$ 
8              else
9                   $C(h, j) \leftarrow C(h+1, (j-1)/2) \cdot B(h, j)$ 
10     else if  $v \leq n/p$  then
11         if  $v$  even then
12              $C(h, v) \leftarrow C(h+1, v/2)$ 
13         else if  $v = 1$  then
14              $C(h, 1) \leftarrow B(h, 1)$ 
15         else
16              $C(h, v) \leftarrow C(h+1, (v-1)/2) \cdot B(h, v)$ 
```

Suma de Prefijos (n,p)

Brent:

$$\frac{W(n)}{p} \leq T(n, p) \leq \frac{W(n)}{p} + T(n)$$

Con $W(n) = O(n)$, $T(n) = \log(n)$, tenemos

$$O(n/p) \leq T(n, p) \leq O(n/p + \log(n))$$

Entonces

$$S(n, p) = \frac{O(n)}{O(n/p + \log(n))}$$

$$E(n, p) = \frac{O(n/p)}{O(n/p + \log(n))} = \frac{O(1)}{O(1 + p \frac{\log(n)}{n})}$$



Caso recursivo: quicksort, suma de prefijos

Quicksort secuencial

Algorithm 4: **Partition**(A, l, r)

Entrada: Arreglo A ; índices l y r

Salida: Posición final del pivote

```
1  $p \leftarrow A[r]$ 
2  $i \leftarrow l - 1$ 
3 for  $j \leftarrow l$  to  $r - 1$  do
4   if  $A[j] \leq p$  then
5      $i \leftarrow i + 1$ 
6     intercambiar  $A[i] \leftrightarrow A[j]$ 
7 intercambiar  $A[i + 1] \leftrightarrow A[r]$ 
8 return  $i + 1$ 
```

Algorithm 5: **Quicksort secuencial**(A, l, r)

Entrada: Arreglo A ; índices l y r

Salida: Arreglo ordenado

```
1 if  $l < r$  then
2    $q \leftarrow \text{Partition}(A, l, r)$ 
3   Quicksort secuencial( $A, l, q - 1$ )
4   Quicksort secuencial( $A, q + 1, r$ )
```

Sea $n = r - l + 1$.

$$T_{\text{part}}(n) = O(n)$$

Si la partición es balanceada, entonces:

$$T_s(n) = 2T_s\left(\frac{n}{2}\right) + O(n)$$

Solución recursiva:

$$T_s(n) = O(n \log n)$$

Quicksort paralelo

Algorithm 6: Partition-Paralelo(A, n)

Entrada: Arreglo $A[1, \dots, n]$

Salida: Arreglo particionado y
posición q del pivote

```
1  $pivot \leftarrow A[n]$ 
2 for  $i = 1$  to  $n - 1$  pardo
3    $B[i] \leftarrow (A[i] < pivot)$ 
4  $P \leftarrow \text{PrefixSum-Paralelo}(B)$ 
5  $k \leftarrow P[n - 1]$ 
6 for  $i = 1$  to  $n - 1$  pardo
7   if  $B[i] = 1$  then
8      $C[P[i]] \leftarrow A[i]$ 
9   else
10     $C[k + 1 + i - P[i]] \leftarrow A[i]$ 
11  $C[k + 1] \leftarrow pivot$ 
12  $q \leftarrow k + 1$ 
13 return  $C, q$ 
```

Algorithm 7: Quicksort-Paralelo(A, n)

Entrada: Arreglo $A[1, \dots, n]$

Salida: Arreglo A ordenado

```
1 if  $n \leq 1$  then
2   return  $A$ 
3  $(A, q) \leftarrow \text{Partition-Paralelo}(A, n)$ 
4 pardo
5   Quicksort-Paralelo( $A[1, \dots, q - 1], q - 1$ )
6   Quicksort-Paralelo( $A[q + 1, \dots, n], n - q$ )
7 return  $A$ 
```

PRAM: Quicksort paralelo - complejidad

Si asumimos particiones balanceadas

$$q \simeq \frac{n}{2}$$

luego:

$$W(n) = 2W(n/2) + \Theta(n) = \Theta(n \log n)$$

Span:

$$T_{\infty}(n) = T_{\infty}(n/2) + \Theta(\log n) = \Theta(\log^2 n)$$

Aplicando Brent:

$$T_p(n, p) = O\left(\frac{n \log n}{p} + \log^2 n\right)$$

PRAM: Suma de prefijos (algoritmo recursivo)

Algorithm 8: Suma de prefijos (algoritmo recursivo)

```
1 Input: vector  $A = [x_1, x_2, \dots, x_n]$  con  $n = 2^k$ 
2 Output: suma de prefijos  $s_i$  para  $1 \leq i \leq n$ 
3 if  $n = 1$  then
4   | set  $s_1 \leftarrow x_1$ ; exit
5 for  $1 \leq i \leq n/2$  pardo
6   | set  $y_i \leftarrow x_{2i-1} \cdot x_{2i}$ 
7 Calcular en forma recursiva, la suma de prefijos de  $[y_1, y_2, \dots, y_{n/2}]$  y almacenarlos
   en  $[z_1, z_2, \dots, z_{n/2}]$ 
8 for  $1 \leq i \leq n$  pardo
9   | if  $i$  is even then
10    | |  $s_i \leftarrow z_{i/2}$ 
11    | else if  $i = 1$  then
12    | |  $s_1 \leftarrow x_1$ 
13    | else
14    | |  $s_i \leftarrow z_{(i-1)/2} \cdot x_i$ 
```

PRAM: Suma de prefijos (algoritmo recursivo)

Suma de prefijos de 8 elementos.

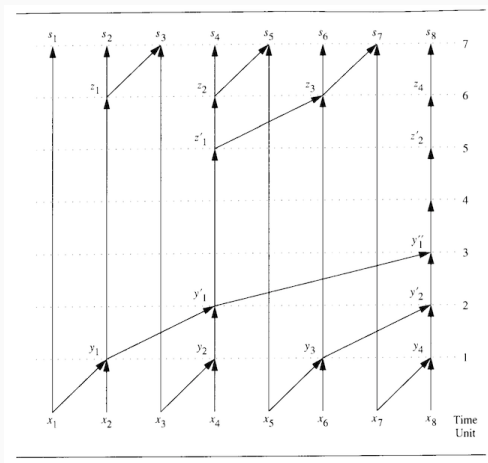


Figura 2: Suma de prefijos de 8 elementos (recursivo) [1] (Fig.2.1 p.47)

PRAM: Suma de prefijos (algoritmo recursivo)

En el paso 1 se ejecuta 1 operación

En los pasos 2 y 4 se ejecuta 1 operación en paralelo

La recursividad tiene una complejidad dada por

$$T(n) = T(n/2) + a$$

, donde a es una constante.

La solución de la recurrencia está dada por: $T(n) = O(\log n)$

Asimismo, dado

$$W(n) = W(n/2) + bn$$

, donde b es una constante.

$$W(n) = W_1(n) + W_2(n) + W_4(n) = O(n)$$

$$\begin{aligned} \text{Dado } f(1) &= 0; f(n) = f(n-1) + n \text{ (reescribir } f(n-1) \text{ como } f(n-1)-1 + n-1) \\ &= f(n-2) + n-1 + n \\ &= f(n-3) + n-2 + n-1 + n \\ &\dots \\ &= f(1) + f(2) + f(3) + \dots + n-1 + n \\ &= n \frac{(n+1)}{2} - 1 = \Theta(n^2) \end{aligned}$$

$$\begin{aligned} f(n) &= f(n/2) + 1 \text{ (reescribir } f(n/2) \text{ como } f((n/2)/2) + 1) \\ &= f(n/4) + 1 + 1 \\ &= f(n/8) + 1 + 1 + 1 \\ &\dots \\ &= f(n/n) + 1 + 1 + \dots + 1 \text{ (log(n) veces)} \\ &= \Theta(\log(n)) \end{aligned}$$

$$\begin{aligned}\text{Dado } f(1) &= 0; f(n) = f(n/2) + n \\ &= f(n/4) + n/2 + n \\ &= f(n/8) + n/4 + n/2 + n \\ &\dots \\ &= f(n/n) + 2 + 4 + \dots + n/4 + n/2 + n \\ &= \Theta(n)\end{aligned}$$

$$\begin{aligned}f(n) &= 2f(n/2) + n \\ &= 4f(n/4) + n + n \\ &= 8f(n/8) + n + n + n \\ &\dots \\ &= nf(n/n) + n + n + \dots + n \text{ (log(n) veces)} \\ &= \Theta(n \log(n))\end{aligned}$$



Joseph Jája *An introduction of parallel algorithms*. University of Maryland (Ch 2.1)

